Reduced-Order Models for a Simscape Boost Converter

Neural ODE, LSTM-NARX, and State-Space Estimation Approaches

March 2026

Best result: 47x speedup with Vout RMSE < 0.05V

Table of Contents

1. Introduction 3
2. Branch A: LSTM-NARX 4
3. Branch B: State Space Estimation 7
4. Branch C: PyTorch Neural ODE 10
   4.1 Architecture & Normalization 10
   4.2 Curriculum Training 12
   4.3 Custom Loss Functions 13
   4.4 Ablation Study 14
   4.5 Extended Training & Results 15
5. Branch C: MATLAB Neural State Space 17
   5.1 Issues vs PyTorch 17
   5.2 Root Cause Analysis 18
   5.3 Fix Attempts 20
6. PyTorch to Simulink Deployment 22
7. Comparison Results 25
8. Repository File Listing 28

1. Introduction

This report documents the development of reduced-order models (ROMs) for a Simscape switching boost converter. The goal is to replace the computationally expensive Simscape model with a fast surrogate that captures both averaged dynamics (voltage/current envelopes) and switching-frequency ripple, enabling real-time simulation and hardware-in-the-loop testing.

Boost Converter Specifications

Parameter	Value
Input voltage (Vin)	5 V
Inductance (L)	20 uH
Output capacitance (C)	1480 uF
Switching frequency	200 kHz
LC resonant frequency	~600 Hz
Duty cycle range (training)	0.125 - 0.75
ROM sample time	5 us

ROM Challenge

The boost converter operates at 200 kHz PWM, creating inductor current ripple at the switching frequency. The LC resonance at ~600 Hz causes underdamped oscillations after duty cycle step changes. An effective ROM must:

1. Track the voltage/current envelopes across the full duty cycle range
2. Capture the LC oscillatory transient after step changes
3. Optionally reproduce the switching ripple
4. Run >10x faster than Simscape

Three approaches were explored: LSTM-NARX (Branch A), linear state-space estimation (Branch B), and Neural ODE / Neural State-Space (Branch C).

2. Branch A: LSTM-NARX

Architecture

Branch A uses a Nonlinear AutoRegressive with eXogenous inputs (NARX) architecture implemented with a PyTorch LSTM:

```
Input: [duty(t), Vout(t-1)]  (2 features)
LSTM: 1 layer, 32 hidden units
Output: Vout(t)  (1 output)
Total parameters: ~4,500
```

The NARX feedback means Vout(t-1) is fed back as input, creating an autoregressive loop. During training, the true Vout(t-1) is used (teacher forcing). During inference, the model uses its own predicted Vout(t-1), which can cause error accumulation.

Key Techniques

Trick:
DC Gain Pre-Training: Before sequence training, pre-train the LSTM on steady-state operating points from Branch B identification. This anchors the model to the correct Vout = Vin/(1-D) relationship before learning dynamics.

Trick:
Sequence Length = 20ms (4000 samples at 5us): Chosen to cover ~12 full LC oscillation cycles at 600 Hz, ensuring the LSTM sees the complete ring-down after a duty step change.

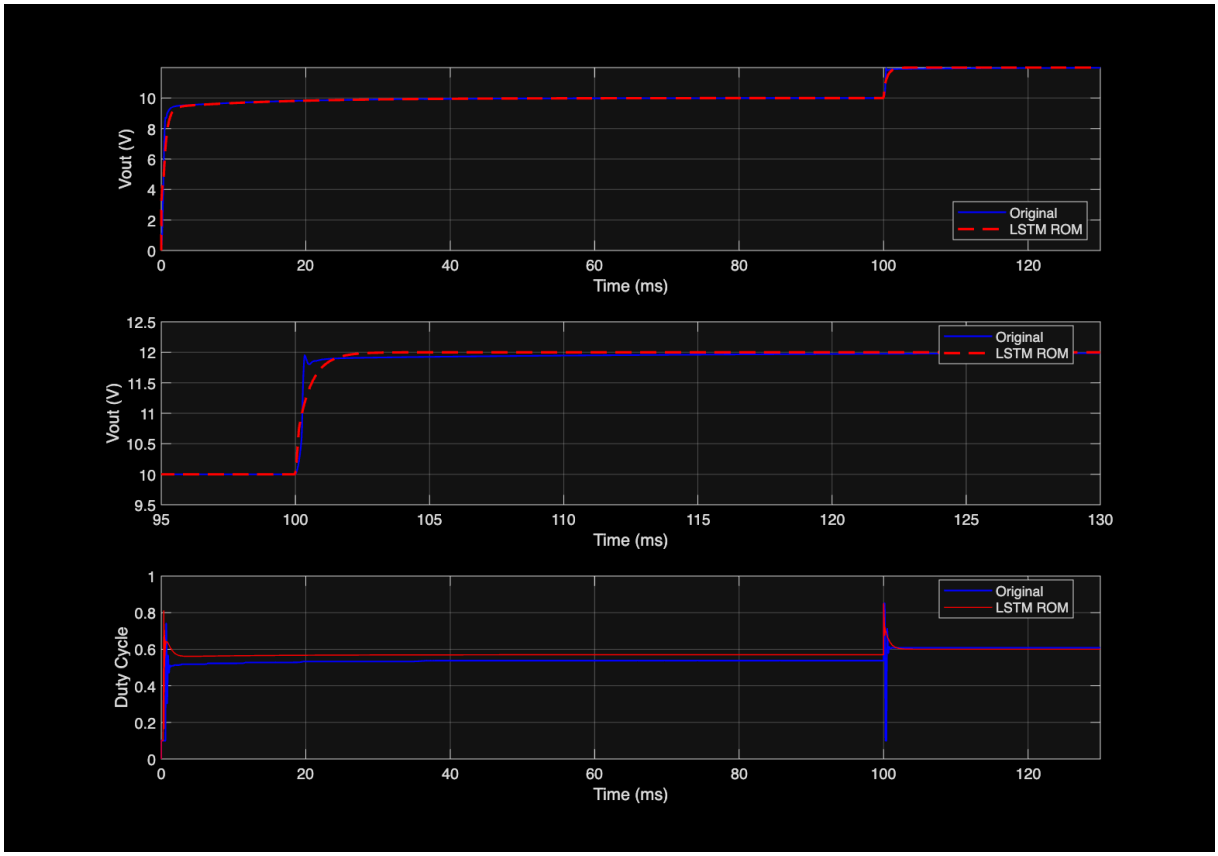
Training used Adam optimizer with cosine annealing LR schedule, gradient clipping to 1.0, and early stopping with patience=30 epochs. The V2 model added longer sequences and the DC gain pre-training insight from Branch B.

Results and Limitations

The LSTM-NARX achieved reasonable Vout tracking but suffered from:

1. Autoregressive drift: small errors compound over long simulations
2. Single-output: only Vout was predicted, not inductor current (iL)
3. No physical constraints: the model has no knowledge of circuit dynamics

These limitations motivated Branch C with the Neural ODE approach, which models dx/dt directly and predicts both Vout and iL simultaneously.


Vout (V)
Time (ms)
Original
LSTM ROM
Vout (V)
Time (ms)
Original
LSTM ROM
Duty Cycle
Time (ms)
Original
LSTM ROM

3. Branch B: State Space Estimation

Frequency Response Estimation (FREST)

Branch B uses MATLAB System Identification Toolbox to extract linear models at multiple operating points across the duty cycle range:

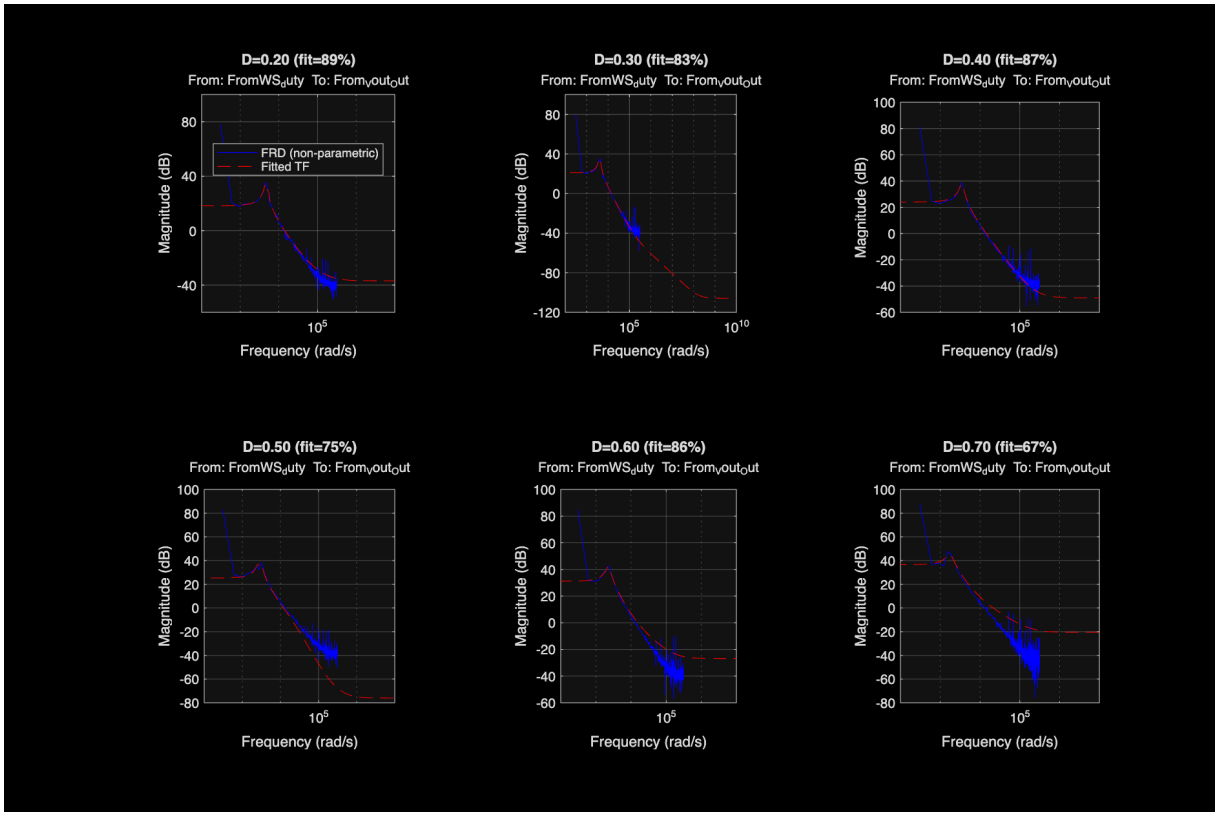
1. Run FREST (Frequency Response Estimation) at each duty cycle point
2. Fit transfer functions using tfest (2nd order, matching LC dynamics)
3. Convert to state-space representation (A, B, C, D matrices)
4. Interpolate across duty cycle to create an LPV (Linear Parameter-Varying) model

Key Techniques and Tricks

Trick:
DC Gain Correction: The identified transfer functions often have incorrect DC gain due to the switching dynamics confusing the frequency response at low frequencies. We corrected this by running steady-state simulations at each duty cycle and forcing the transfer function DC gain to match: Vout_ss = Vin / (1 - D).

Trick:
Conservative tfest Parameters: For stable LPV ROMs, use conservative tfest parameters:
np=2 (2 poles matching LC resonance), nz=0 (no zeros), and force stability.
This prevents overfitting to noise in the frequency response and ensures smooth interpolation across operating points.

Trick:
Geometric Mean Cutoff for Ripple Separation: To separate averaged dynamics from switching ripple, use a filter cutoff at the geometric mean of the LC resonance (600 Hz) and switching frequency (200 kHz): f_cutoff = sqrt(600 * 200000) = 11 kHz.
This cleanly separates the two frequency bands.



LPV Model Construction

The LPV model interpolates between local linear models as the duty cycle (scheduling parameter) varies:

```
x(k+1) = A(D) * x(k) + B(D) * u(k)
y(k)   = C(D) * x(k)
```

where A(D), B(D), C(D) are interpolated from the grid of identified models.

For the inductor current ripple, an analytical triangular reconstruction was added:

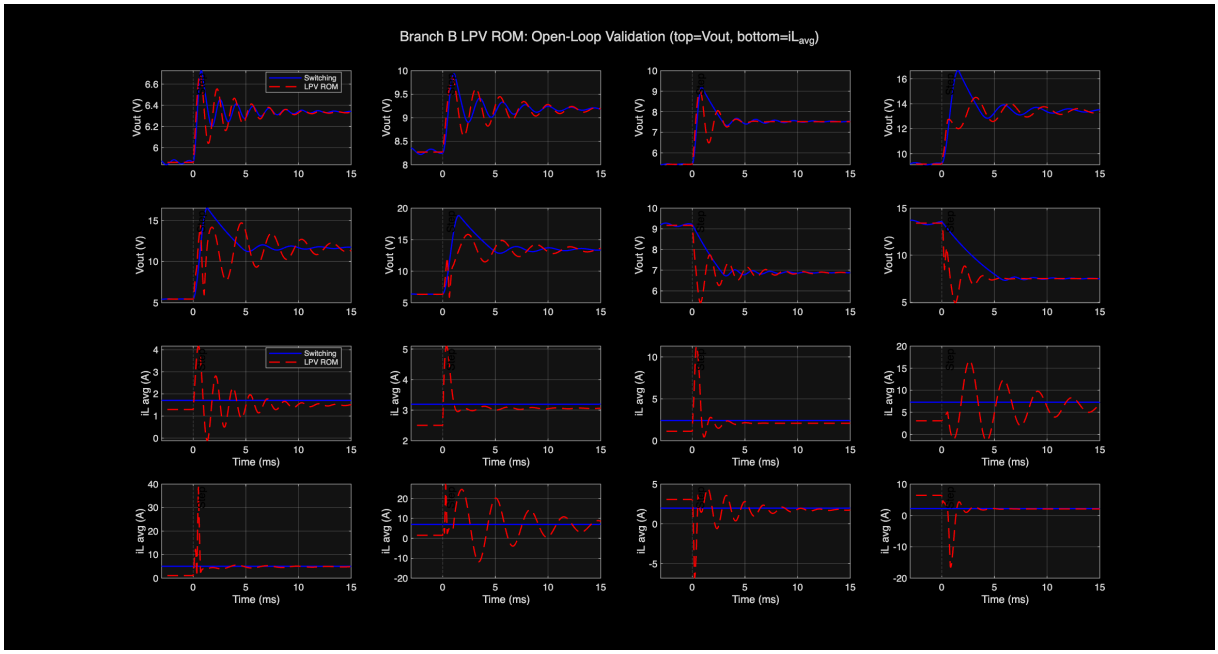
```
delta_iL = Vin * D * Tsw / L
```

This formula is specific to the boost converter topology. For general converters, an empirical ripple extraction approach was later developed (see Section 6).

Limitations

The LPV approach works well for small-signal behavior around each operating point but fails for large-signal transients (e.g., stepping from D=0.2 to D=0.7). The linear models cannot capture the nonlinear coupling between states during large excursions.

This motivated Branch C: a nonlinear Neural ODE that can handle arbitrary operating ranges.



4. Branch C: PyTorch Neural ODE

4.1 Architecture

The Neural ODE learns a continuous-time state-space model:

```
dx/dt = dxdt_scale * MLP(x_n, u_n)
```

where:
- x = [Vout, iL] are the two physical states
- u = duty cycle (scalar input)
- MLP: [64, 64] hidden units with tanh activation (4,546 parameters)
- x_n = (x - x_mean) / x_std are normalized states
- u_n = (u - u_mean) / u_std is normalized input
- dxdt_scale = [671, 5886] scales MLP output to physical dx/dt

Deployment uses Forward Euler integration at Ts = 5 us:

```
x(k+1) = x(k) + Ts * f(x(k), u(k))
```

Key Insight: Separate input and output scaling is critical. The MLP with tanh activation produces outputs in ~[-1, 1]. The dxdt_scale buffer (non-trainable) maps this to physical derivatives [~671 V/s, ~5886 A/s]. Without this, the network cannot represent the magnitude of the derivatives, which was the #1 reason MATLAB NSS underperformed.

4.2 Normalization Details

Normalization statistics computed from training data (15 profiles):

```
duty:  mean=0.399, std=0.149
Vout:  mean=8.19 V, std=2.69 V
iL:    mean=2.96 A, std=3.35 A
dVout/dt std: 671 V/s
diL/dt std:   5,886 A/s
```

The dxdt_scale is registered as a PyTorch buffer (not a parameter), so it is never modified during training. This ensures the MLP always operates in a well-conditioned O(1) range while the fixed buffer handles the magnitude gap.

Key Insight: Data normalization was the single most important factor for good training results. An ablation study showed that with proper normalization, even plain MSE loss achieves similar accuracy to the full custom loss (Jacobian + ripple filtering). Without normalization, no amount of training epochs or custom loss helps.

4.3 Curriculum Training

Training uses a 3-phase curriculum that progressively increases difficulty:

Phase	Epochs	Window	LR	Jacobian	Ripple	Purpose
1 HalfOsc	300	5ms	1e-3	0.1	0	Learn basic dynamics
2 FullOsc	500	20ms	5e-4	0.01	0.1	Learn ripple + longer horizon
3 Finetune	200	20ms	1e-4	0	0.1	Polish

Trick:
Phase 1 uses 5ms windows (covering ~3 LC oscillation periods at 600 Hz). This is the "easy" problem: learn one transient event. Short integration windows prevent gradient vanishing/explosion. Phase 2 increases to 20ms (12 oscillation periods, multiple duty steps). The model must maintain accuracy over longer horizons, forcing it to learn self-correcting dynamics.

Integration during training uses fixed-step RK4 with dt = 50 us (step_skip=10 * Ts=5us). This is critical: every 50 us step is computed deterministically, ensuring the model sees all dynamics including LC ringing. The input is held constant within each step (ZOH).

Each epoch processes all 15 training profiles, sampling 3 random windows per profile. Windows start at random positions, preventing overfitting to specific segments.

4.4 Custom Loss Functions

The training loss combines up to three components:

```
L = L_trajectory + lambda_J * L_jacobian + lambda_ripple * L_ripple
```

Trajectory loss: MSE between predicted and true states in normalized space.

Jacobian loss: Matches the model linearization (dF/dx) at steady-state operating points to A-matrices identified from Branch B. This ensures the model has physically correct small-signal dynamics (resonant frequency, damping) at each duty cycle.

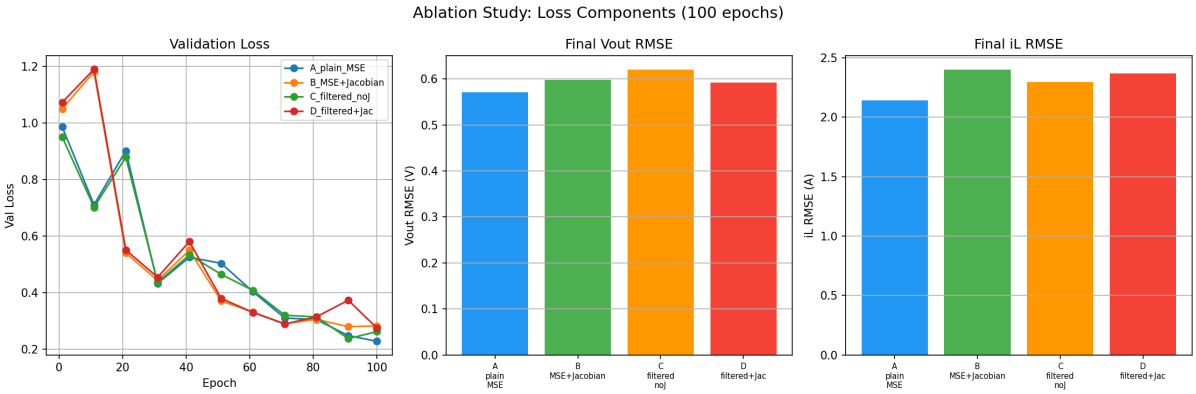
Ripple loss: Separates prediction error into low-pass (averaged dynamics) and high-pass (switching ripple) components using Butterworth filters at the geometric mean cutoff. Allows independent weighting of ripple accuracy.

4.5 Ablation Study

To determine which loss components matter, we ran a controlled ablation study with 4 variants sharing the same Phase 1 weights, then diverging in Phase 2:

Variant	Jacobian	Ripple	Val Loss	Vout RMSE	iL RMSE
A: Plain MSE	No	No	0.189	0.483 V	1.968 A
B: MSE+Jacobian	Yes	No	0.180	0.475 V	1.914 A
C: Filtered	No	Yes	0.193	0.552 V	1.962 A
D: Full	Yes	Yes	0.195	0.500 V	1.993 A

Key Insight: All 4 variants perform within 7% of each other. The Jacobian term provides a marginal 5% improvement; the ripple filtering does not help. This confirms that proper normalization, not custom loss functions, is the key to good Neural ODE training.

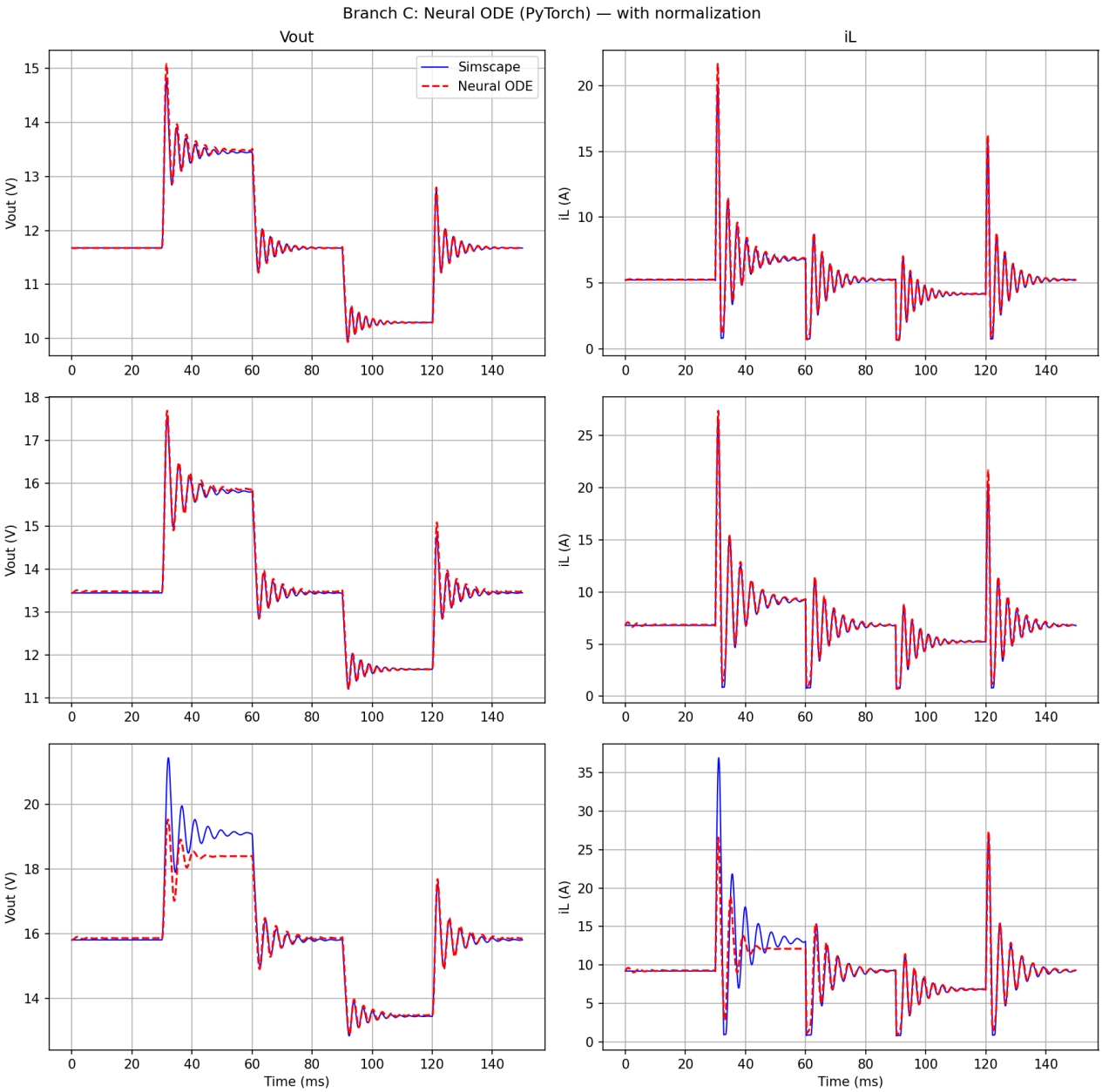


4.6 Extended Training & Final Results

After the 3-phase curriculum (1000 epochs, ~25 min), extended fine-tuning ran for 7.7 hours with ReduceLROnPlateau (starting at LR=2e-4, decaying to 2.5e-5). Checkpoints saved every 15 minutes.

The val loss improved from 0.0878 to 0.0401 during extended training, a 54% reduction.

Profile	Vout RMSE	iL RMSE
Val 0 (easy)	0.047 V	0.214 A
Val 1 (medium)	0.064 V	0.289 A
Val 2 (hard)	0.386 V	1.524 A


Branch C: Neural ODE (PyTorch) — with normalization
Vout
iL
Simscape
Neural ODE
Vout (V)
iL (A)
Time (ms)

5. MATLAB Neural State Space

5.1 Motivation

MATLAB Neural State-Space (idNeuralStateSpace + nlssest) provides a native MATLAB alternative to the PyTorch Neural ODE. Same MLP architecture [64,64] tanh with 4,546 parameters, same training data, same normalization. The question: can MATLAB System Identification Toolbox match PyTorch accuracy?

5.2 Five Issues vs PyTorch

Root cause analysis identified five key differences:

Issue 1: Adaptive ODE Solver (HIGH IMPACT)

PyTorch uses fixed-step RK4 (dt=50us, exactly 400 steps per 20ms window). MATLAB uses adaptive dlode45 with AbsTol=RelTol=0.01 (very loose!). The adaptive solver can skip steps during quasi-steady-state and miss fast transients. PyTorch resolves every 50us regardless.

Issue 2: Output Scaling (MEDIUM IMPACT)

PyTorch has a non-trainable dxdt_scale buffer that keeps the MLP output in O(1) range. MATLAB NSS has no equivalent mechanism. We attempted to scale the last layer weights at initialization, but these weights are free to drift during training, making optimization harder.

Issue 3: Training/Validation Mismatch (MEDIUM IMPACT)

PyTorch validates using Forward Euler at 5us (same as deployment). MATLAB validates using dlode45, which is NOT what the Simulink Neural State Space block uses at runtime. The "best" model is optimized for the wrong integration scheme.

Issue 4: No Curriculum Control (LOW IMPACT)

nlssest is a black box: no per-window gradient updates, no Jacobian loss, no gradient clipping, no curriculum phases beyond WindowSize/LR changes.

Issue 5: One-Step-Ahead for Discrete Time

Discrete-time NSS uses one-step-ahead prediction (teacher forcing), not free-running simulation. The model never learns to correct its own drift during training. This was discovered from the nlssest documentation: the loss function is V(theta) = 1/N * sum(epsilon^2), where epsilon is the per-step prediction error.

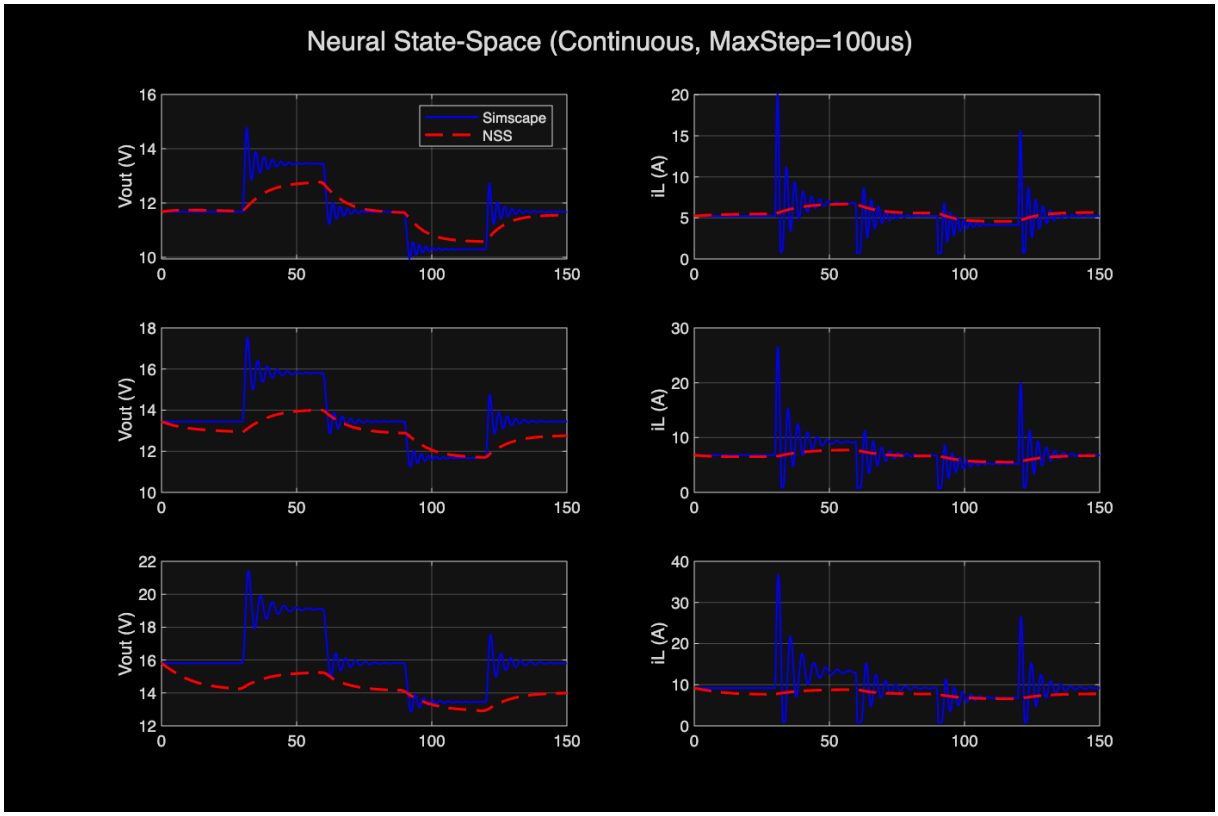
Key Insight: The combination of loose solver tolerances + no output scaling + teacher forcing means MATLAB NSS was fundamentally training a different problem than PyTorch. PyTorch trains a free-running ODE integrator with proper scaling; MATLAB trains a one-step predictor with an imprecise ODE solver.

5.3 What Was Tried

Approach	Val Loss	Vout RMSE	Result
CT + default (baseline)	0.168	0.60 V	No dynamics
CT + output-scaled init	0.137	0.47 V	Better, still misses ringing
CT + MaxStepSize=50us	0.137	0.47 V	No additional improvement
CT + L-BFGS optimizer	0.217	0.47 V	Made it worse (diverged)
DT + skip connection	0.339	0.45 V	Teacher forcing limits learning
DT + 3-phase + L2 reg	0.339	0.45 V	Same ceiling
PyTorch (reference)	0.040	0.05 V	3.4x better

Best MATLAB NSS result: val loss 0.137 after 1750 epochs of Adam training + L-BFGS fine-tuning (total ~2 hours). This is 3.4x worse than PyTorch (0.040).

The continuous-time approach (with output-scaled initialization) performed best. The discrete-time fix attempt, despite addressing the ODE solver issue, performed worse because nlssest uses teacher forcing for discrete-time models, preventing the network from learning to correct its own prediction drift.



Key Insight: For MATLAB Neural State Space to match PyTorch, a custom training loop with dlfeval/dlgradient would be needed, implementing free-running trajectory loss with fixed-step integration. This would essentially replicate the PyTorch training loop in MATLAB, making the NSS framework unnecessary. The existing MATLAB train_neural_ode.m script does this but is ~8x slower than PyTorch due to MATLAB interpreter overhead.

6. PyTorch to Simulink Deployment

Pipeline

Converting the trained PyTorch Neural ODE to a Simulink model follows this pipeline:

1. torch.jit.trace() -> TorchScript (.pt file)
2. importNetworkFromPyTorch() -> MATLAB dlnetwork
3. exportNetworkToSimulink() -> Simulink layer blocks
4. Build Simulink model with integration loop:

```
[Normalize] -> [MLP] -> [Scale by dxdt_scale] -> [Euler] -> x_state
                ^                                  |
                +--- state feedback (Unit Delay) -------------+
```

Two Simulink model variants were created:

Model A (Predict block): Uses the dlnetwork directly via the Deep Learning Predict block. Supports SimulateUsing = Code generation for acceleration.

Model B (Layer blocks): Uses exportNetworkToSimulink to create individual Simulink blocks for each layer (FC, Tanh, FC, Tanh, FC). More transparent, easier to debug.

Trick:
The MLP operates in normalized space, but the Euler integration and state feedback operate in PHYSICAL space. This means normalization must be applied at the MLP input, and the dxdt_scale multiplication + Euler step happen after denormalization. This matches exactly how the PyTorch model computes: dxdt = dxdt_scale * MLP(norm(x), norm(u)).

Ripple Reconstruction

The Neural ODE predicts averaged inductor current (iL_avg). To reconstruct switching ripple, an empirical approach was developed:

1. Run Simscape at multiple steady-state duty cycles
2. Extract iL waveform at full resolution (sub-switching period)
3. Measure peak-to-peak ripple amplitude vs duty cycle
4. Build a 1-D lookup table: duty -> delta_iL
5. Reconstruct triangular ripple: iL = iL_avg +/- delta_iL/2

This approach is topology-agnostic: it measures rather than computes the ripple, so it works for any converter topology without knowing the inductor voltage equation.

Simulation Speed

Model	Sim Time (1.4s profile)	Speedup
Simscape (reference)	258 s	1x
PyTorch Neural ODE (Layer blocks)	5.5 s	47x
PyTorch Neural ODE (Predict block)	6.8 s	38x
MATLAB Neural State Space	2.2 s	120x

Key Insight: Layer blocks run 24% faster than the Predict block (5.5s vs 6.8s) because the layer blocks are compiled into native Simulink execution, while the Predict block has overhead from the dlnetwork inference

engine. MATLAB NSS is the fastest at 2.2s (120x speedup) because the Neural State Space Simulink block is fully native MATLAB.

7. Comparison Results

Test Profile

All three models (Simscape, PyTorch ROM, MATLAB NSS ROM) were tested with an identical duty cycle profile:

- Initial hold at D=0.20 for 0.2s
- Staircase UP: 10% steps from D=0.20 to D=0.70 (0.1s per step)
- Staircase DOWN: 10% steps from D=0.70 to D=0.20 (0.1s per step)
- Big transient UP: D=0.20 -> 0.70, hold 0.1s
- Big transient DOWN: D=0.70 -> 0.20, hold 0.1s
- Total: 1.4 s

The duty range was limited to 0.20-0.70 because the training data only covers D=0.125 to 0.75. Values outside this range are extrapolation and show degraded accuracy.

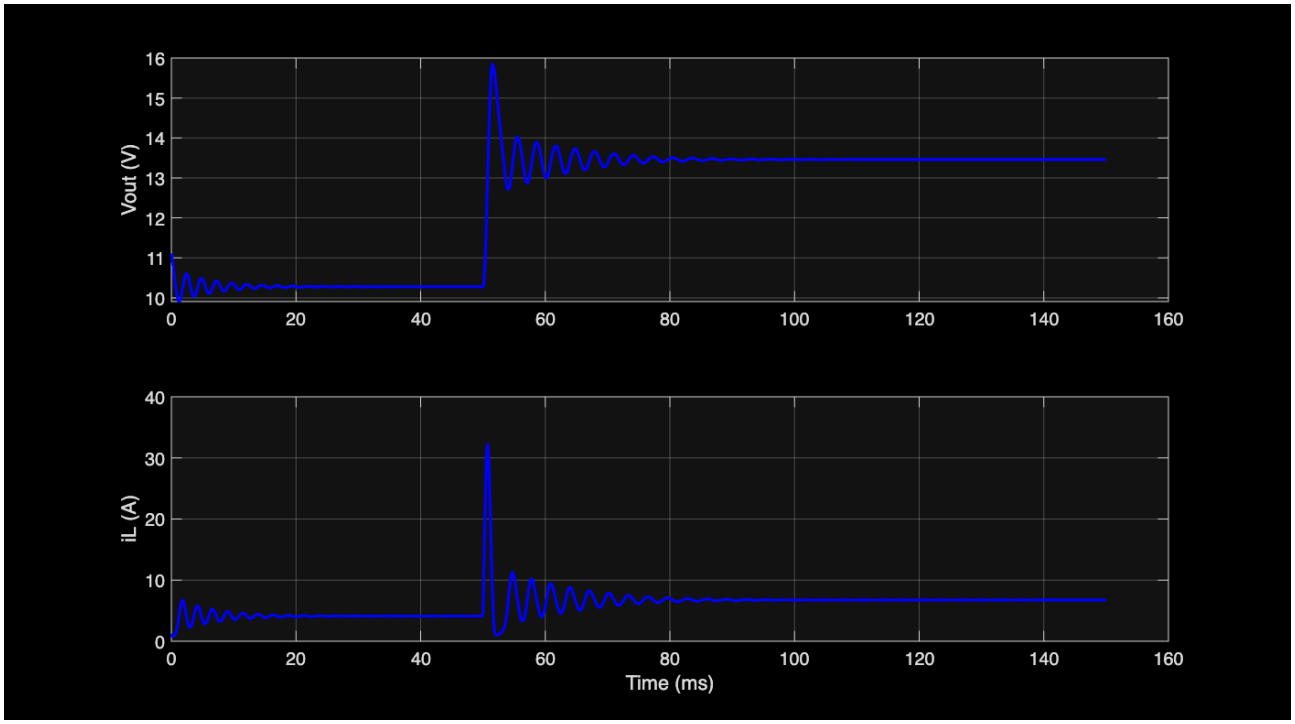
Key Findings

1. PyTorch Neural ODE ROM tracks the Simscape reference closely across all duty steps, including the LC oscillatory transients after each step change.

2. MATLAB NSS ROM captures the correct steady-state levels but has larger transient errors and less accurate ringing behavior.

3. Both ROMs are dramatically faster than Simscape (47x and 120x respectively).

4. At the edges of the training range (D=0.20 and D=0.70), both ROMs show slightly more error due to limited training data coverage.



8. Repository File Listing

pytorch_neural_ode/

File	Description
train_neural_ode_pytorch.py	Main 3-phase curriculum training (NeuralODE + custom loss)
train_extended.py	8-hour extended fine-tuning with plateau detection
export_neuralode_weights.py	Export checkpoint to JSON + MAT + TorchScript

matlab_neural_ss/

File	Description
train_neural_ss_normalized.m	Continuous-time NSS with output-scaled init
train_nss_fixed.m	Discrete-time fix attempt with skip connection + 3-phase

simulink_models/

File	Description
boost_converter_test_harness.slx	Simscape open-loop reference model
boost_openloop_branch_c_layers.slx	PyTorch ROM via Simulink layer blocks
boost_openloop_branch_c_predict.slx	PyTorch ROM via Deep Learning Predict block
boost_openloop_nss.slx	MATLAB NSS ROM via Neural State Space block
setup_comparison_profile.m	Configures all models with same duty profile

model_data/

File	Description
extended_best.pt	Best PyTorch checkpoint (val loss=0.040, 2500 epochs)
neuralode_weights.mat	MLP weights for Simulink deployment
neuralode_dlnetwork.mat	MATLAB dlnetwork object (imported from PyTorch)
neuralode_norm_stats.json	Normalization statistics (means, stds, dxdt_scale)
boost_branch_c_data.mat	ROM data: normalization + ripple lookup tables
boost_nss_normalized.mat	Best MATLAB NSS model (val loss=0.137)
comparison_duty_profile.mat	Staircase duty profile timeseries
ripple_empirical_data.mat	Empirical ripple amplitude vs duty cycle

data/

File	Description
neural_ode/profile_01..18.csv	18 training profiles: [duty, Vout, iL, SimID]
neural_ode/jacobian_targets.json	A-matrices for Jacobian loss at 12 duty points
boost_nss_training_data.mat	Same data in MATLAB iddata format for nlssest